

Technical Report: Web-Based Integer Factorization System

Algorithm Analysis and System Architecture

May 22, 2026

Abstract

This technical report provides a comprehensive analysis of a web-based integer factorization application. The system is designed to factor large integers directly in the browser, leveraging modern web technologies including JavaScript, Web Workers for multi-threading, and WebAssembly (Wasm) for near-native computational performance. The core factorization engine implements the Multiple Polynomial Quadratic Sieve (MPQS), augmented with advanced techniques such as the Double Large Prime variation, fast modular arithmetic using floating-point decomposition, and hybrid sparse-dense linear algebra over $\text{GF}(2)$. Supporting algorithms include deterministic and probabilistic Miller-Rabin primality tests, Pollard's Rho algorithm for factoring residuals, and the Tonelli-Shanks algorithm for modular square roots. This document details the mathematical foundations, algorithmic logic, data structures, and execution flows of the provided codebase.

Contents

1	Introduction	3
2	System Architecture and Control Flow	3
2.1	User Interface and Orchestration	3
2.2	Main Factorization Loop	3
3	Mathematical Preliminaries	4
3.1	Notation	4
3.2	Smooth Numbers	5
4	Primality Testing	5
4.1	Deterministic Miller-Rabin	5
4.2	WebAssembly 64-bit Probable Prime	5
5	Auxiliary Algorithms	5
5.1	Perfect Power Detection	5
5.2	Pollard's Rho Algorithm	5
5.3	Modular Square Root: Tonelli-Shanks	6

6	The Quadratic Sieve Algorithm	6
6.1	Parameter Selection and Factor Base	6
6.2	Multiple Polynomial Generation (MPQS)	6
6.3	Fast Modular Arithmetic	7
6.4	Sieving Process	7
6.5	Large Prime Variations	7
7	Linear Algebra over GF(2)	7
7.1	Sparse-Dense Hybrid Strategy	8
7.2	BitSet and GF(2) Operations	8
8	Factor Discovery	8
9	Conclusion	8

1 Introduction

Integer factorization—the decomposition of a composite integer into a product of smaller integers—is a fundamental problem in number theory and cryptography. While factoring small integers is trivial, the difficulty of factoring large semiprimes underpins the security of cryptosystems like RSA.

The provided codebase implements a robust, client-side integer factorization tool. It operates entirely within a web browser, utilizing HTML/CSS for the user interface and a sophisticated combination of JavaScript and WebAssembly for the computational backend. The primary algorithm employed is the Quadratic Sieve (QS), which is the fastest known algorithm for factoring integers under 100 decimal digits, and it remains competitive up to about 130 digits.

This report is structured as follows: Section 2 outlines the high-level system architecture and control flow; Section 3 covers mathematical preliminaries; Sections 4 through 7 provide deep dives into each algorithmic component; and Section 9 concludes the report.

2 System Architecture and Control Flow

The application consists of an HTML frontend (`index.html`) and several JavaScript/Wasm modules.

2.1 User Interface and Orchestration

The frontend provides a dark-themed, responsive interface. The user inputs an integer, which can be in standard notation or using the caret operator (e.g., 2^{4*31}). The input is parsed using a sanitization wrapper around `eval`, automatically converting literals to `BigInt`.

The factorization process is offloaded to a Web Worker to prevent blocking the main UI thread. The worker code is dynamically constructed as a Blob, importing the main Quadratic Sieve module.

2.2 Main Factorization Loop

The orchestration logic in `index.html` implements a recursive decomposition strategy to completely factor the input integer N . The algorithm maintains a stack of composite factors to process.

Algorithm 1 Integer Factorization Orchestration

```
1: Input: Integer N
2: Output: List of prime factors
3: factors[N]
4: primeFactors[]
5: if N<0 then primeFactors.push(1)
6: end if
7: while factors is not empty do
8:   afactors.pop()
9:   if a==1 then
10:     continue
11:   end if
12:   risPerfectPower(a)
13:   if isPrime(a) then
14:     primeFactors.push(a)
15:   else if r =1 then
16:     firoot(a,r)
17:     for i=1 to r do
18:       factors.push(f)
19:       aa/f
20:     end for
21:   else
22:     fQuadraticSieveFactorization(a)
23:     factors.push(f)
24:     factors.push(a/f)
25:   end if
26: end while
27: return sorted primeFactors
```

▷ Runs in Web Worker

This loop ensures that perfect powers are broken down into their roots immediately, primes are identified and collected, and composites are split using the Quadratic Sieve until only prime factors remain.

3 Mathematical Preliminaries

3.1 Notation

- N : The integer to be factored.
- p : A prime number.
- B : The smoothness bound.
- F : The factor base, a set of primes pB such that $(\frac{N}{p})=1$ (N is a quadratic residue mod p).
- $L(N)=\exp(\ln N \ln \ln N)$: The standard sub-exponential notation.

3.2 Smooth Numbers

An integer is B -smooth if all of its prime factors are less than or equal to B . The Quadratic Sieve relies on finding B -smooth numbers to construct congruences of squares.

4 Primality Testing

Before attempting to factor a number, the system verifies its compositeness. The system uses two variants of the Miller-Rabin primality test.

4.1 Deterministic Miller-Rabin

For general BigInts, the code implements the Miller-Rabin strong pseudoprime test. It writes $N-1 = 2^r d$ with d odd. It tests bases a to check if:

- $a^d \not\equiv 1 \pmod{N}$, or
- $a^{2^j d} \not\equiv -1 \pmod{N}$ for some $0 \leq j < r$.

If N is less than specific thresholds, a deterministic set of bases is sufficient. The `getBases` function hardcodes these thresholds up to 3,317,044,064,679,887,385,961,981. For larger N , it falls back to generating primes up to a bound proportional to $\ln N \ln \ln N$, which yields a high probability of correctness.

4.2 WebAssembly 64-bit Probable Prime

For 64-bit integers (specifically when handling double large prime residuals in QS), the code uses a pre-compiled WebAssembly module exporting `isProbablyPrime64`. This provides extremely fast probabilistic checks necessary for the sieve's smooth relation processing.

5 Auxiliary Algorithms

5.1 Perfect Power Detection

A number a is a perfect power if $a = m^r$

for integers m and r . The `isPerfectPower(a)` function in `index.html` iterates r from 2 up to the bit-length of a . For each r , it computes the integer r -th root using Newton's method (`iroot`) and checks if the result raised to the power r equals a .

5.2 Pollard's Rho Algorithm

Pollard's Rho is a probabilistic algorithm for integer factorization, particularly efficient for finding small factors. In this system, it is compiled to WebAssembly (`PollardsRho64`) and used within the Quadratic Sieve to factor residuals that consist of two large primes (double large primes). It uses Brent's cycle detection variant with the polynomial $f(x) = x^2 + c \pmod{p}$.

5.3 Modular Square Root: Tonelli-Shanks

The Tonelli-Shanks algorithm solves $x^2 \equiv n \pmod{p}$ for prime p . It is crucial for generating the factor base and initializing the sieve.

1. Find Q and S such that $p-1=2^s Q$ with Q odd.
2. Find a quadratic non-residue z modulo p .
3. Set $c \equiv z^Q \pmod{p}$, $R \equiv n^{(Q+1)/2} \pmod{p}$, $t \equiv n^Q \pmod{p}$, and $M=S$.
4. While $t \not\equiv 1 \pmod{p}$:
 - Find the lowest i such that $t^{2^i} \equiv 1 \pmod{p}$.
 - Let $b \equiv R^{2^{M-i}} \pmod{p}$.
 - Update $R \leftarrow Rb \pmod{p}$, $t \leftarrow t b^2 \pmod{p}$, $M=i$.

The implementation includes an optimization for $p \equiv 3 \pmod{4}$, where the root is simply $n^{(p+1)/4} \pmod{p}$.

6 The Quadratic Sieve Algorithm

The Quadratic Sieve is the central algorithm of the application. It finds two integers X and Y such that $X^2 \equiv Y^2 \pmod{N}$, which implies N divides $(X-Y)(X+Y)$. If $X \not\equiv \pm Y \pmod{N}$, then $\gcd(X-Y, N)$ yields a non-trivial factor.

6.1 Parameter Selection and Factor Base

The smoothness bound B is chosen dynamically based on $L(N)$: $B = \max(512, \min(\text{limit}, 624 L(N)))$

The factor base F consists of primes $p \leq B$ for which N is a quadratic residue. The system also evaluates a multiplier k (1 to 150) to find kN that maximizes the number of small quadratic residues, thereby increasing sieve yield.

6.2 Multiple Polynomial Generation (MPQS)

Using a single polynomial $Q(x) = x^2 - N$ limits the sieve range. The implementation uses the Multiple Polynomial variant. A polynomial is defined as: $Q(x) = Ax^2 + 2Bx + C$

where A is a product of primes in F , and B is chosen such that $B^2 \equiv N \pmod{A}$, which implies $C = (B^2 - N)/A$. The roots modulo primes in F are found using Hensel's lifting and the Chinese Remainder Theorem (`squareRootsModuloOddPrimesProduct`).

When $N \equiv 1 \pmod{8}$, a special Q

form is used where A is odd, and the polynomial evaluated is $Q(x) = 4A^2x^2 + 4A(2Ax+B)^2 - N$

, ensuring integer values and better sieving properties.

6.3 Fast Modular Arithmetic

Computing $X \bmod p$ for large BigInts X is a bottleneck. The Quadratic Sieve implements `FastModBigInt`, which splits a BigInt into 51-bit chunks. It then uses floating-point arithmetic to compute the modulus: $X = \sum_{i=0}^n x_i 2^{51i}$

By precomputing $2^{51} \bmod p$ and using IEEE 754 double precision arithmetic, the modulo operation is performed significantly faster than native BigInt modulo.

6.4 Sieving Process

Sieving is performed over intervals (segments) using a byte array. For each prime p in the factor base, the sieve adds $\log_2(p)$ to the sieve array at indices corresponding to the roots of $Q(x) \equiv 0 \pmod{p}$.

To accelerate this, the sieving logic is offloaded to WebAssembly. The Wasm module exports:

- `singleBlockSieve`: Updates sieve values for medium and large primes.
- `findSmoothEntry`: Scans the sieve array for bytes exceeding a threshold.
- `handleSmallWheels`: Processes small primes using highly optimized wheel logic.

6.5 Large Prime Variations

A number is considered smooth if its remaining residual after dividing by the factor base is small.

- **Single Large Prime**: If the residual r is less than B^2 , it is assumed to be prime. The system stores partial relations and matches them by r . When two relations share the same large prime, they are multiplied to form a full smooth relation.
- **Double Large Prime**: For very large N , residuals up to B^3 are considered. If the residual $r < 2^{64}$, `isProbablyPrime64` checks if it's prime. If not, `PollardsRho64` factors it into $r = p_1 p_2$. These are modeled as edges in a graph. A cycle in this graph yields a valid smooth relation.

The cycle detection for double large primes uses a Union-Find data structure (Graph class). When the number of edges plus components minus vertices exceeds the required number of relations, a Depth-First Search (`iterateCycles`) extracts the cycles and generates the corresponding congruences.

7 Linear Algebra over $\text{GF}(2)$

Once sufficient smooth relations are found, the system constructs a matrix over $\text{GF}(2)$ where each row represents a relation, and each column represents the parity of a prime in the factor base. A solution corresponds to a linear dependency (null space vector), implying a subset of relations whose product yields a perfect square.

7.1 Sparse-Dense Hybrid Strategy

For large numbers, the matrix is extremely sparse. The code implements a hybrid approach:

1. **Sparse Collection:** Relations are collected until the number of rows exceeds the number of columns by a margin (32).
2. **Sparse Elimination:** The SparseGF2Matrix stores rows and columns as arrays of indices. It applies Markowitz-like heuristics:
 - Eliminates rows with only one non-zero entry (guarantees a dependency or simplifies the matrix).
 - Deletes columns with high density to prevent fill-in.
3. **Dense Elimination:** The remaining dense block is converted to a BitSet representation. Standard Gaussian elimination is performed to find the null space.

7.2 BitSet and GF(2) Operations

The BitSet class uses Uint32Array and BigUint64Array for memory-efficient bitwise operations. The xor method leverages 64-bit XOR operations to process 64 bits at a time, drastically speeding up row reduction.

8 Factor Discovery

After the linear algebra phase produces a solution, the corresponding relations are combined. Let the solution consist of relations $X_i^2 \equiv Y_i \pmod{N}$. We compute: $X = X_i \pmod{N}$

$$Y = Y_i \pmod{N}$$

The square root of Y_i

is computed efficiently by halving the exponents of the factor base primes. Finally, $\gcd(XY, N)$ is calculated. If it is a non-trivial factor (not 1 or N), the factorization succeeds. If not, the algorithm requests another dependency from the solver.

9 Conclusion

The analyzed codebase presents a highly optimized, state-of-the-art implementation of the Multiple Polynomial Quadratic Sieve tailored for the web environment. By intelligently combining JavaScript's BigInt capabilities, Web Workers for concurrency, and WebAssembly for performance-critical loops (sieving, modular arithmetic, Pollard's Rho), the system achieves remarkable factorization speeds directly in the browser. The inclusion of advanced techniques like the Double Large Prime variation, fast floating-point modular reduction, and hybrid sparse-dense linear algebra demonstrates a deep understanding of computational number theory and modern web engineering.

References

- [1] R. D. Silverman, *The Multiple Polynomial Quadratic Sieve*, Mathematics of Computation, Vol. 48, No. 177 (1987), pp. 329-339.
- [2] S. Contini, *Factoring Integers with the Self-Initializing Quadratic Sieve*, Master's thesis, University of Georgia, 1997.
- [3] C. Pomerance, *The Quadratic Sieve Factoring Algorithm*, Advances in Cryptology – EUROCRYPT '84, Springer, 1985.
- [4] A. Tonelli, *Bemerkung über die Auflösung quadratischer Congruenzen*, Nachrichten von der Königl. Gesellschaft der Wissenschaften und der Georg-Augusts-Universität zu Göttingen, 1891.